PEARLS AQI PREDICTOR

Air Quality Index Forecasting System

End-to-End Machine Learning & MLOps Project Documentation

Project Type	Machine Learning / Data Science / MLOps
Application	Interactive AQI prediction dashboard
Frontend / Deployment	Streamlit web application
Forecast Objective	AQI prediction for the next 3 days
Submission Basis	Provided Pearls AQI Predictor project brief
Project Developer	Soojal Kumar
Live Application	https://soojalkumar-ai-aqi-predictor-streamlit-apphome-lf7r23.streamlit.app/

Prepared for project submission

Date: 27 August 2026

Table of Contents

1. Executive Summary

Pearls AQI Predictor is an end-to-end machine learning project designed to forecast Air Quality Index (AQI) for a city over the next three days. The supplied project brief defines a serverless forecasting system combining external weather and pollutant data, feature engineering, historical backfilling, machine-learning training, automated pipeline execution, and an interactive web dashboard.

The application is presented through a Streamlit interface. The objective is to transform changing environmental observations into model-ready features and expose current and forecasted air-quality information through a simple dashboard. The brief also calls for time-based features such as hour, day, and month, derived features such as AQI change rate, evaluation using RMSE, MAE, and R^2, explainability using SHAP or LIME, and alerts for hazardous AQI levels.

This document translates those requirements into a submission-ready technical report. Where the supplied brief does not specify an exact implementation choice, that item is described as an architectural option rather than an unverified implementation fact.

2. Project Background and Problem Statement

Air quality varies with pollutant concentration, weather conditions, time, and local environmental activity. A dashboard that only reports current AQI provides limited forward-looking information. This project therefore focuses on forecasting future AQI so users can understand expected air-quality conditions before they occur.

Problem Statement: Build an automated, scalable, and interactive AQI forecasting system that obtains environmental data, transforms it into useful machine-learning features, trains and evaluates forecasting models, and presents real-time and forecasted AQI information through a web dashboard.

The project brief requires an end-to-end AQI prediction system, a scalable automated pipeline, an interactive dashboard for real-time and forecasted AQI data, and detailed documentation of the achieved work.

3. Objectives

- Collect raw weather and pollutant observations from an external environmental API.
- Transform raw observations into model inputs and target variables through feature engineering.
- Include temporal information such as hour, day, and month and derive indicators such as AQI change rate.
- Backfill historical data to create training data.
- Train and compare suitable forecasting models using a reproducible pipeline.
- Evaluate candidate models using RMSE, MAE, and R^2.
- Store reusable features in a feature-store architecture and trained models in a model registry where supported.
- Automate recurring data and model workflows.
- Provide an interactive Streamlit dashboard for current conditions and forecasts.
- Add model explainability and hazardous-AQI alerts.

4. Requirements and Compliance Mapping

Project Brief Requirement	Documentation / System Response
Fetch raw weather and pollutant data	External environmental/weather API ingestion layer.
Compute model inputs and outputs	Feature-engineering stage creates predictors and targets.
Time-based features	Hour, day, month and other temporal variables.

Project Brief Requirement	Documentation / System Response
AQI change rate	Derived trend/change-rate feature.
Feature Store	Centralized storage layer for engineered features.
Historical backfill	Historical execution creates training data.
Train/evaluate models	Training pipeline compares candidate models.
RMSE, MAE, R^2	Primary regression evaluation metrics.
Model Registry	Versioning/retrieval layer for trained model.
Hourly feature job	Automation schedule for feature refresh.
Daily training job	Automation schedule for model retraining.
Interactive dashboard	Streamlit prediction and visualization interface.
SHAP/LIME	Explainability layer.
Hazard alerts	AQI warning mechanism.

5. System Architecture

The system follows a modular MLOps architecture. Data acquisition is separated from feature engineering; feature generation is separated from model training; model artifacts are separated from the presentation layer; and recurring jobs are orchestrated by automation.

Architecture: External APIs → Data Ingestion → Feature Engineering → Feature Store → Historical Backfill → Model Training/Evaluation → Model Registry → Streamlit Dashboard. Automation controls the recurring feature and training workflows.

Layer	Responsibility
External APIs	Weather and pollutant observations
Data Ingestion	Fetch, validate, normalize and persist raw data
Feature Pipeline	Temporal + derived features and targets
Feature Store	Reusable model-ready feature storage
Historical Backfill	Generate past feature/target records
Training Pipeline	Train, compare and evaluate models
Model Registry	Version and retrieve production model
Automation / CI-CD	Hourly feature job + daily training job
Streamlit App	Load model/features and display predictions
Explainability + Alerts	SHAP/LIME insights and hazardous-AQI warnings

6. Data Pipeline and Feature Engineering

The feature pipeline fetches raw weather and pollutant information, computes model inputs and targets, and stores resulting features in a feature-store layer. The same feature logic should be reused for historical backfill so training and inference remain consistent.

Candidate features: pollutant measurements, weather observations, hour, day, month, lag/trend variables, and derived indicators such as AQI change rate.

Data quality: validate missing values, timestamps, duplicates, unexpected ranges, schema consistency, and API failures before data enters the model-ready layer.

7. Historical Data Backfill

The project brief requires the feature script to be executed across historical dates to generate the feature/target dataset used by the machine-learning models.

- Select an appropriate historical date range.
- Run the same feature-generation logic used by the live pipeline.
- Generate predictor features and future AQI targets.
- Validate chronological ordering and remove or flag unusable records.
- Store the resulting training dataset.
- Use time-aware validation to reduce future-data leakage.

8. Machine Learning Training Pipeline

The training pipeline reads historical features and targets, trains candidate models, evaluates them, and stores the selected model in a model registry. The supplied brief specifically recommends Scikit-learn models such as Random Forest and Ridge Regression and permits TensorFlow/PyTorch experimentation.

Training procedure: Load data → define targets → chronological split → train candidate models → evaluate → select best model → persist artifact → register version.

9. Model Evaluation

Metric	Purpose	Interpretation
MAE	Average absolute prediction error	Lower is better; directly interpretable in AQI units.
RMSE	Penalizes larger errors	Lower is better; useful when large misses matter.
R^2	Variance explained by the model	Higher is better; closer to 1 indicates stronger fit.

The final submission should record the actual selected model, validation period, MAE, RMSE, and R^2 from the project run. The supplied brief defines the metrics but does not provide numerical results.

10. Automation and CI/CD

The supplied brief requires recurring automation: the feature script should run every hour and the training script every day. GitHub Actions and Apache Airflow are suggested options.

Hourly: Data ingestion → feature generation → validation → feature-store update.

Daily: Training data refresh → model training → evaluation → model registration.

Operational controls: logging, retries, failure notifications, timeout handling, secret management, and model-version tracking.

11. Streamlit Web Application

The project is deployed as a Streamlit web application. The dashboard is responsible for loading the production model and required features, generating predictions, and presenting current and forecasted AQI information.

Live Application: https://soojalkumar-ai-aqi-predictor-streamlit-apphome-lf7r23.streamlit.app/

- Display latest AQI and relevant environmental information.
- Show forecasted AQI for the next three days.
- Provide trend visualizations.
- Use clear labels and units.
- Highlight hazardous conditions.
- Keep inference features consistent with training features.

12. Explainable AI and AQI Alerts

The brief recommends SHAP or LIME for feature-importance explanations. Explainability can help users understand which variables contribute to predictions.

- Use SHAP or LIME for global and, where practical, local explanations.

- Present explanations in readable dashboard language.
- Trigger visible warnings when forecasted AQI reaches a hazardous category.
- Use the AQI classification standard adopted by the implemented application.

13. Technology Stack

Layer	Technology / Approach
Programming	Python
Machine Learning	Scikit-learn; optional TensorFlow/PyTorch
Web Dashboard	Streamlit
Data Sources	External weather and air-pollution API
Feature Management	Feature Store architecture
Model Lifecycle	Model Registry
Automation	GitHub Actions or Apache Airflow
Explainability	SHAP or LIME
Deployment	Streamlit-hosted web application

14. Functional Requirements

- FR-01: Obtain environmental/weather observations.
- FR-02: Transform observations into model-ready features.
- FR-03: Support temporal and derived features.
- FR-04: Generate historical feature/target data.
- FR-05: Train and evaluate candidate forecasting models.
- FR-06: Calculate MAE, RMSE and R^2.
- FR-07: Persist/version the selected model.
- FR-08: Expose predictions through an interactive dashboard.
- FR-09: Present the forecast horizon.
- FR-10: Provide hazardous-AQI alerts.
- FR-11: Provide SHAP/LIME explanations where implemented.
- FR-12: Support recurring automated execution.

15. Non-Functional Requirements

- Scalability: modular components that can scale independently.
- Reliability: logging and recoverable failures.
- Performance: responsive dashboard inference.
- Maintainability: separation of data, feature, model and UI logic.
- Security: credentials stored as secrets/environment variables.
- Reproducibility: dependencies, model versions, data windows and metrics recorded.
- Usability: clear AQI values, forecasts, charts and alerts.

16. Testing and Validation Strategy

Area	Validation
API ingestion	Response handling, timeout, missing fields and malformed records.
Data quality	Nulls, duplicates, timestamps, ranges and schema.
Feature engineering	Calculation correctness and leakage checks.
Model training	Successful training and reproducible metrics.
Model evaluation	Correct MAE, RMSE and R^2 calculations.
Inference	Training and deployment feature-schema compatibility.
Dashboard	Inputs, charts, forecast display and alert behavior.
Automation	Triggering, logging and failure handling.
Deployment	Live application opens and core functions work.

Actual test results and numerical model metrics should be inserted from project run logs/notebooks. The provided brief does not contain those values.

17. Deployment

The user-facing project is deployed as a Streamlit web application. The deployment target provides a browser-accessible interface for the prediction dashboard.

- Install project dependencies.
- Configure API/data-store credentials as deployment secrets.
- Load the approved model and feature configuration.
- Start the Streamlit application.
- Validate data access and inference.
- Monitor availability and update the model when a new version is approved.

18. Limitations and Risks

- Forecasts depend on external data quality and availability.
- Weather-pollution relationships vary by season and location.
- A model trained on one city or period may not generalize elsewhere.
- API outages can interrupt real-time data acquisition.
- A forecast is not a guarantee of future AQI.
- Model performance can degrade due to data drift.
- SHAP/LIME explain model behavior but do not establish causality.

19. Future Enhancements

- Add prediction intervals/probabilistic forecasting.
- Support multiple cities.
- Compare additional time-series and boosting models.
- Add data/model drift monitoring.

- Trigger retraining based on performance degradation.
- Add configurable hazardous-AQI notifications.
- Improve mobile accessibility.
- Integrate additional environmental signals.
- Add model comparison and rollback.

20. Conclusion

Pearls AQI Predictor demonstrates the structure of a complete machine-learning application rather than a standalone prediction notebook. The project brief combines data ingestion, feature engineering, historical backfill, model training and evaluation, model management, automation, explainability, and an interactive Streamlit dashboard into one end-to-end workflow.

The architecture demonstrates practical Data Science and MLOps skills: transforming external data into reusable features, evaluating forecasting models with regression metrics, automating recurring workflows, and delivering predictions through a web interface.

21. References

1. Provided project brief: *Pearls AQI Predictor*. The brief specifies the feature pipeline, historical backfill, training pipeline, automation, Streamlit web application, EDA, forecasting-model experimentation, explainability, alerts, and final submission requirements.

2. Live application supplied for this submission:
https://soojalkumar-ai-aqi-predictor-streamlit-apphome-lf7r23.streamlit.app/

Developer: Soojal Kumar

Source note: The supplied project brief is the authoritative source for project requirements. Exact model names, numerical results, API provider, feature-store product, registry product, and CI/CD provider should only be stated as final implementation facts when confirmed by project source code or run artifacts.